



# 7. Neural Networks: Regression

*The Boston Housing dataset that needs Neural Network to deal with?*

Previous Section:

 [6. Neural Network: Multi-Class Classification](#)

Contents:

[1. The Boston Housing Dataset](#)

[2. Build the Regression Model](#)

[Summary](#)

[References](#)

Let's clear something up first. Neural Networks don't suddenly "break" when you move from classification to regression. They just stop pretending everything is a probability.

Because up until now, you've seen them behave like this: "Is this a cat?"  $\rightarrow 0.8$ ; "Is this a dog?"  $\rightarrow 0.2$

Everything is a probability. Everything is squeezed into a range. Everything feels, nicely contained. But what happens when the question changes? ***"What is the price of this house?"*** Now suddenly, 0.8 means nothing. You don't want confidence. You want a number.

And that's where things start to feel a bit uncomfortable. Activation functions that once made perfect sense—Sigmoid, Softmax—now feel restrictive. Why force an output into a probability range when the answer could be 250,000, or 1,200,000?

So here's the shift—and it's an important one: In regression, the Neural Network is no longer trying to choose between options. It is trying to estimate a

continuous value.

No classes. No probabilities. Just a prediction. And that leads to a very natural question: ***"What does the output layer even look like now?"*** Is it still using an activation function? Or is it just a raw number?

Hold that thought. Because to answer it properly, we need a real problem to anchor everything. And there's probably no better place to start than this one: The Boston Housing dataset

## 1. The Boston Housing Dataset

Let's step into a world where Neural Networks stop classifying, and start *estimating*.

Now we're finally stepping into something real. Not theory. Not "what if." → An actual dataset with an actual problem: ***"Given everything we know about a house... can we predict its price?"***

And this is where the **Boston Housing dataset** quietly becomes one of the most important starting points for regression. Because unlike classification, nothing here is cleanly separated. No labels like *spam / not spam*. No categories like *cat / dog*. Just numbers influencing other numbers.

**The Boston Housing Dataset:** Think of this dataset as a collection of houses across Boston suburbs. Each house is described by a set of features: Crime rate in the area; Average number of rooms; Distance to employment centers; Accessibility to highways, and more

And then one final value: **Median house price**. That's it. That's the target. So the Neural Network is no longer asking: *"Which class does this belong to?"*.

Instead, it asks something much more subtle: *"Given all of this... what should the price be?"*

### Loading the Dataset

```

from tensorflow.keras.datasets import boston_housing

# Load the boston housing dataset
(x_train, y_train), (x_test, y_test) = boston_housing.load_data()

# Display shape
print('Training set shape:', x_train.shape)
print('Testing set shape:', y_train.shape)
print('Train targets shape:', x_test.shape)
print('5 first target  :', y_test[:5])

```

Training set shape: (404, 13)

Testing set shape: (404,)

Train targets shape: (102, 13)

5 first target : [ 7.2 18.8 19. 27. 22.2]

At this point, everything looks fine. But there's a hidden problem.

**The Subtle Problem: Different Scales.** Not all features speak the same "language."

- Crime rate might be something like  $0.1 \rightarrow 80$
- Number of rooms might be  $1 \rightarrow 10$
- Distance could be  $1 \rightarrow 12$

Now imagine feeding this directly into a Neural Network. What happens? The model starts *thinking* that features with larger values are more important even if they're not. And that's dangerous.

**So we fix it with Normalization**

```

# Find mean and standard deviation
mean = x_train.mean(axis=0)
std = x_train.std(axis=0)

# Normalize the data
x_train = (x_train - mean) / std
x_test = (x_test - mean) / std

# Print the mean and std of the normalized data

print('Mean:', x_train.mean())
print('Std :', x_train.std())

```

Mean: 1.0146745084159847e-17  
Std : 1.0000000000000002

We bring everything to the same scale using **Z-score normalization**:

- Center the data around 0
- Scale it to have standard deviation = 1

In simple terms: "Make every feature comparable."

And the result?  $Mean \approx 0 \rightarrow Std \approx 1$  → Exactly what we want.

**Now, a very important realization:** You might be thinking: *"Wait... that's it?"* → "No encoding? No fancy preprocessing?" → ***"The Neural Network is just ready?"***

And the answer is: **Yes and no.** Yes — because this dataset is already numerical. There are no categorical features to encode. But also no — because what you just did is more important than it looks. You didn't just "normalize data" → You made learning *possible*.

- Without normalization: Training becomes unstable → Gradients behave unpredictably → The network struggles to converge
- With normalization: Everything becomes smooth → Learning becomes efficient → The network can actually *understand relationships*

So now, we're finally at the starting line. The data is clean. The features are aligned. And the Neural Network? It's no longer trying to classify. It's about to do something much harder: ***Learn how numbers shape other numbers.***

## 2. Build the Regression Model

Now that we understand what a neural network is doing conceptually—layers transforming raw numbers into meaningful representations—we can finally implement it for a regression problem.

Since this is the **Boston Housing Dataset**, where we need to predict prices. The Neural Network no longer produces probabilities for categories like “cat = 0.6” or “dog = 0.2” or “bird = 0.2”.

Instead, the model outputs a continuous value → a number. That changes some important things:

- Output layer → **1 neuron (no activation)**
- Loss function → **Mean Squared Error (MSE)**
- Metric → **Mean Absolute Error (MAE)**

### Model Definition

```
from tensorflow import keras
from tensorflow.keras import layers

# Build the model

def build_model():
    hidden_layer1 = layers.Dense(64, activation='relu')
    hidden_layer2 = layers.Dense(64, activation='relu')
    output_layer = layers.Dense(1)

    model = keras.Sequential([hidden_layer1, hidden_layer2, output_layer])
    model.compile(optimizer='rmsprop', loss='mse', metrics=['mae'])
    return model

# Create a model
model = build_model()
```

### Training the Model (50 Epochs, Batch Size 16)

Now we train the model and track MAE per epoch.

```
# Training the Model (50 Epochs, Batch Size 16)
history = model.fit(x_train, y_train, epochs=50, batch_size
=16,
                    validation_split=0.2, verbose=1)
```

### Extract MAE Across Epochs

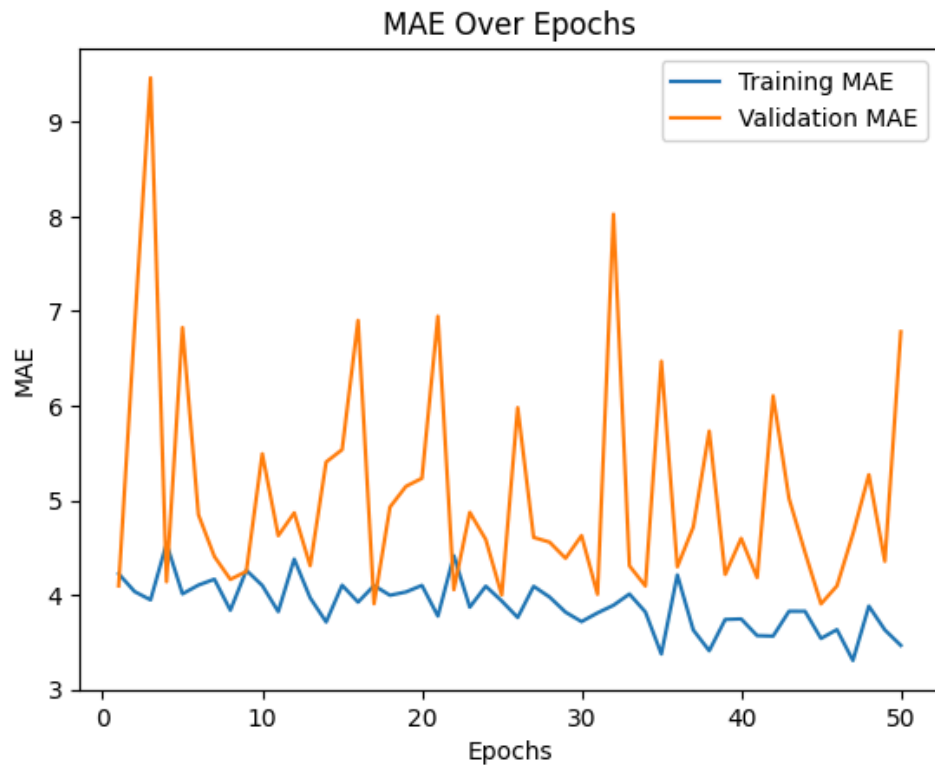
Now we analyze how the model improves over time.

```
# Extract MAE Across Epochs
import matplotlib.pyplot as plt

mae = history.history['mae']
val_mae = history.history['val_mae']

epochs = range(1, len(mae) + 1)

plt.plot(epochs, mae, label='Training MAE')
plt.plot(epochs, val_mae, label='Validation MAE')
plt.xlabel('Epochs')
plt.ylabel('MAE')
plt.legend()
plt.title('MAE Over Epochs')
plt.show()
```



### Mean MAE Calculation

We also compute the average error over training:

```
import numpy as np

# Mean MAE Calculation
mean_mae = np.mean(mae)
print("Mean Training MAE:", mean_mae)
```

Mean Training MAE: 3.8943840885162353

And for validation:

```
# Mean MAE Calculation for validation
mean_val_mae = np.mean(val_mae)
print("Mean Validation MAE:", mean_val_mae)
```

Mean Validation MAE: 5.065217866897583

## What is Actually Happening Under the Hood?

Now let's step back from code and see what is really happening at every epoch:

1. The model makes predictions: "This house costs \$250k"
2. It compares with real values: Error = difference between prediction and reality
3. That error becomes a signal: "Adjust weights slightly"
4. The network slowly reshapes itself: Better patterns → better predictions

So the full loop is: **Data** → **Prediction** → **Error** → **Weight Update** → **Improved Model**

## Cross Validation (Preview of Next Post)

Right now, we used a simple validation split. But in real machine learning systems, this is not enough. Because a single split can be biased. So in the next post, we introduce: **Cross Validation for Neural Networks**

Where instead of trusting one split, we: Train multiple models → Rotate validation sets → Average performance. This gives a much more stable understanding of how well the model truly learns.

## Summary

Neural Networks for regression are one of the most direct applications of deep learning. We take structured data, pass it through multiple dense layers, and let the model learn relationships between features and a continuous output.

In this case:

- Input: Housing features
- Output: House price (continuous value)
- Loss: MSE
- Metric: MAE

But the most important idea is not the code itself. It is the fact that the model is not memorizing prices—it is learning the relationship between features and value.

And that is the core idea of regression in neural networks: ***Simple input*** → ***layered transformation*** → ***numeric prediction***.

## References

[https://d2l.ai/chapter\\_linear-regression/index.html](https://d2l.ai/chapter_linear-regression/index.html)

<https://www.kaggle.com/code/zahrazolghadr/neural-network-model-for-regression>

## Next Section:

 [8. Cross-Validation in Neural Networks](#)